Student Name: Shiva Sai Krishna S | **Register Number:** 1911260318

Course: CSA0421 - Operating System | **GitHub Repo:** krishshiva721/CSA04--Assignment1

TASK 10: PERFORMANCE MEASUREMENT & BENCHMARKING

Empirical System Call Overhead and Process vs Thread Profiling

1. MEASURED PERFORMANCE RESULTS (SYSTEM TIME & USER TIME)

Execution Context	User Mode Time (s)	Kernel Mode Time (s)	Total Real Time (s)	System Call Traps
Copy (Buffer = 1 Byte)	0.012 s	0.036 s	0.048 s	10,005 calls
Copy (Buffer = 4096 Bytes)	0.000 s	0.000 s	0.0003 s	8 calls
Creation of 1,000 Processes (fork)	0.018 s	0.084 s	0.105 s	1,000 fork calls
Creation of 1,000 Threads (pthread_create)	0.004 s	0.009 s	0.013 s	1,000 clone calls

2. EMPIRICAL EVALUATION AND THEORETICAL ALIGNMENT

The measured metrics strongly validate our theoretical evaluations in Task 6 and Task 7. Increasing the read buffer size to align with the OS hardware page size (4096 bytes) reduced kernel-mode CPU consumption from 0.036 s to negligible levels by minimizing trap overhead. Furthermore, process creation proved ~8.1× slower than thread creation.

3. ALTERNATIVE CONCEPT: PROCESS VS. THREAD BENCHMARK COMPLEXITY

> **Alternative Scenario: Benchmark Creation of 1,000 Processes vs 1,000 Threads**
>
> - **Memory Space Allocation:** 1,000 processes consume $1,000 \times \text{Page Table Overhead}$ (~40 MB dedicated space). 1,000 threads share the single parent page table array, consuming only virtual stack reservations (~8 MB shared).
> - **Time Complexity:** `pthread_create` executes in $O(1)$ constant time allocation per thread (~9 µs), whereas `fork()` requires $O(M)$ address space duplication per process (~105 µs).